Lab 04 – Partial Reconfiguration (DFX) with MicroBlaze

Final summary · Nexys A7-100T (xc7a100tcsg324-1) · Vivado 2025.2 / Vitis Unified IDE 2025.2

Result	Value
Demo	Modules A → B → C → A swapped while the MicroBlaze kept running ✔
Bitstreams	Full 3,825,904 B · each partial 799,693 B (20.9 % of the full bitstream)
Timing at 100 MHz	Met in all three configurations – WNS 1.861 ns, WHS 0.035 ns, 0 failing endpoints
Utilization	1,634–1,650 LUTs (2.6 %) · 1,764 FFs (1.4 %) · 32 BRAM (23.7 %) · 20 IOB
Power	0.232 W total on-chip (identical in all configurations)
Verification	336-line full-cycle log: counter continuous 0x0000–0x014F, 0 wrong outputs

1. Objective

Lab 04 divides the FPGA into a **static region** and one **reconfigurable partition**. The static region holds the MicroBlaze system and is configured once. The partition holds a small 16-bit module that is exchanged at run time by loading a **partial bitstream**, while the processor keeps running and reports over the UART which module is currently loaded.

Reused from Lab 03:

- The complete MicroBlaze system (MicroBlaze, 128 KB LMB BRAM, Clocking Wizard, reset, AXI UART Lite at 9600 baud, interrupt controller, MDM), opened from `lab03.xpr` and saved as the new project `C:\AHE\lab04`.
- One AXI GPIO, reconfigured as the interface to the partition; the second GPIO and the PWM IP were removed.
- Pin assignments of clock, reset, UART and the 16 LEDs; the bare-metal flow with `Xil_In32` / `Xil_Out32`.

2. Hardware design

In this lab the block-design wrapper is not the top module. A hand-written VHDL top level `top.vhd` instantiates the block design as `u_sys` and the reconfigurable module as `u_rm`. The module output also drives LD0–LD15.

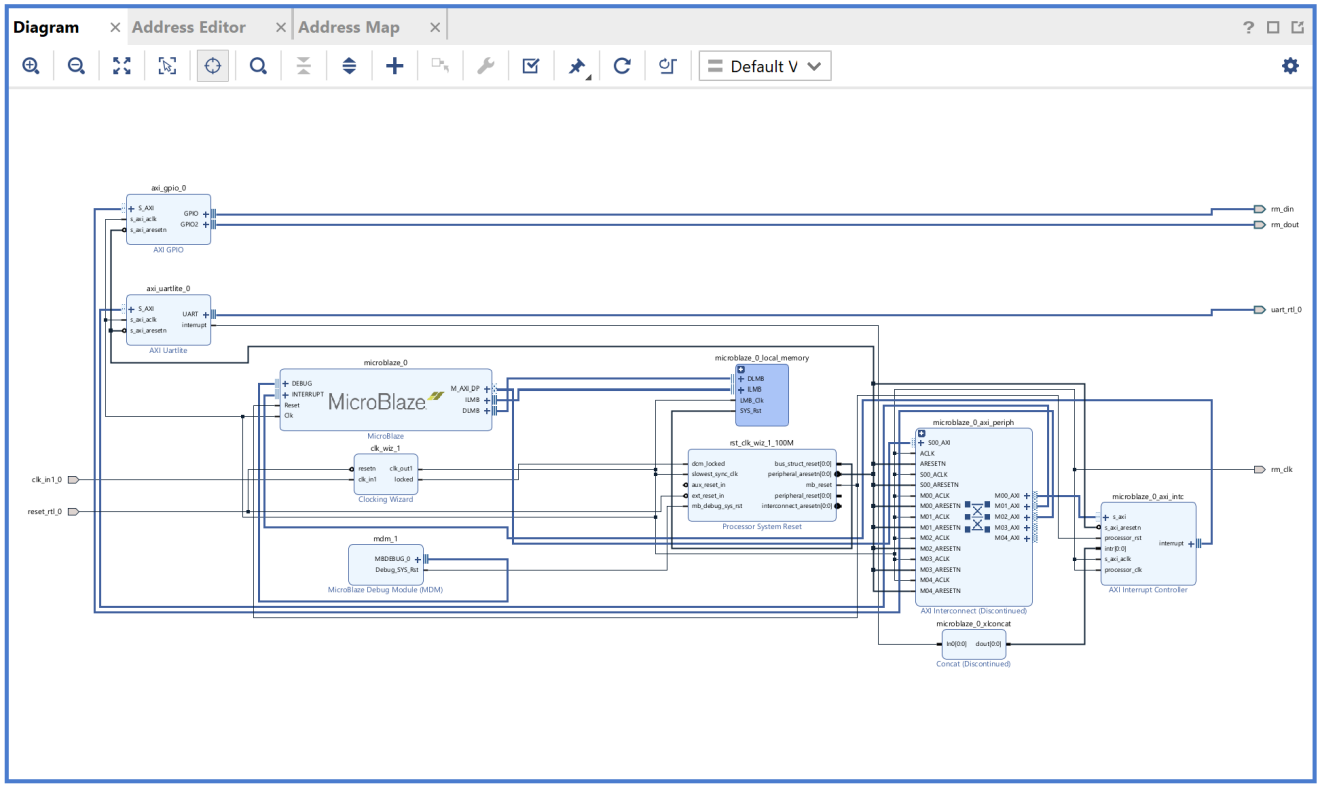


Figure 1 – Block design (static region). axi_gpio_0 channel 1 drives rm_din, channel 2 reads rm_dout; rm_clk is the 100 MHz system clock.

Top level and interface

Instance / port	Direction	Description
u_sys : design_microblaze_wrapper	–	Static region: MicroBlaze, memory, UART, GPIO, clock and reset
u_rm : rm_a (rm_b, rm_c)	–	Reconfigurable partition, partition definition rm_pd
`rm_clk`	static → RM	100 MHz clock from the Clocking Wizard
`rm_din_tri_o[15:0]`	static → RM	GPIO channel 1, all outputs (offset 0x00)
`rm_dout_tri_i[15:0]`	RM → static	GPIO channel 2, all inputs (offset 0x08); also LED[15:0]

Reconfigurable modules

Module	Function (registered on rising_edge(clk))	Answer to test value 0x0001	Logic
rm_a	`dout <= din;` – pass-through	0x0001	16 FF, 0 LUT
rm_b	`dout <= not din;` – invert	0xFFFE	16 FF, 16 LUT
rm_c	`dout <= din(7 downto 0) & din(15 downto 8);` – swap bytes	0x0100	16 FF, 0 LUT

Address map (from top.xsa)

Peripheral	Base address	High address	Use in software
axi_gpio_0	0x4000_0000	0x4000_FFFF	din (0x00, output), dout (0x08, input)
axi_uartlite_0	0x4060_0000	0x4060_FFFF	xil_printf, 9600 baud
microblaze_0_axi_intc	0x4120_0000	0x4120_FFFF	Not used
Local memory (I/D)	0x0000_0000	0x0001_FFFF	Code and data, 128 KB

3. DFX set-up and floorplan

- **Tools → Enable Dynamic Function eXchange**, then **Create Partition Definition** on `u_rm` (definition `rm_pd`, first module rm_a). rm_b and rm_c added in the DFX wizard.
- **Configurations:** config_1 = rm_a, config_2 = rm_b, config_3 = rm_c. **Runs:** parent `impl_1` (config_1) and the children `child_0_impl_1` (config_2) and `child_1_impl_1` (config_3), which reuse the locked static routing of the parent.
- **Pblock** `pblock_u_rm` in clock region X1Y3, full height of the region (Y150–Y199) so that whole configuration frames belong to the partition. `RESET_AFTER_RECONFIG` resets the module after loading; `SNAPPING_MODE ON` aligns the Pblock to frame boundaries.
- DRC (default and dfx_floorplan_checks) is clean after the PCIE site was added to the Pblock (see section 9).

```
create_pblock pblock_u_rm
add_cells_to_pblock [get_pblocks pblock_u_rm] [get_cells -quiet [list u_rm]]
resize_pblock [get_pblocks pblock_u_rm] -add {SLICE_X52Y150:SLICE_X81Y199 DSP48_X1Y60:DSP48_X2Y79
                                             PCIE_X0Y0:PCIE_X0Y0 RAMB18_X1Y60:RAMB18_X2Y79
                                             RAMB36_X1Y30:RAMB36_X2Y39}
set_property RESET_AFTER_RECONFIG true [get_pblocks pblock_u_rm]
set_property SNAPPING_MODE ON [get_pblocks pblock_u_rm]
```

Pblock resource	Range	Amount reserved	Used by largest module
Slices	X52–X81, Y150–Y199	1,500 (6,000 LUT, 12,000 FF)	16 LUT, 16 FF (rm_b)
RAMB36 / RAMB18	X1–X2, Y30–39 / Y60–79	20 / 40	0
DSP48	X1–X2, Y60–79	40	0
PCIE	X0Y0	1 (required by DRC HDPR-45)	0

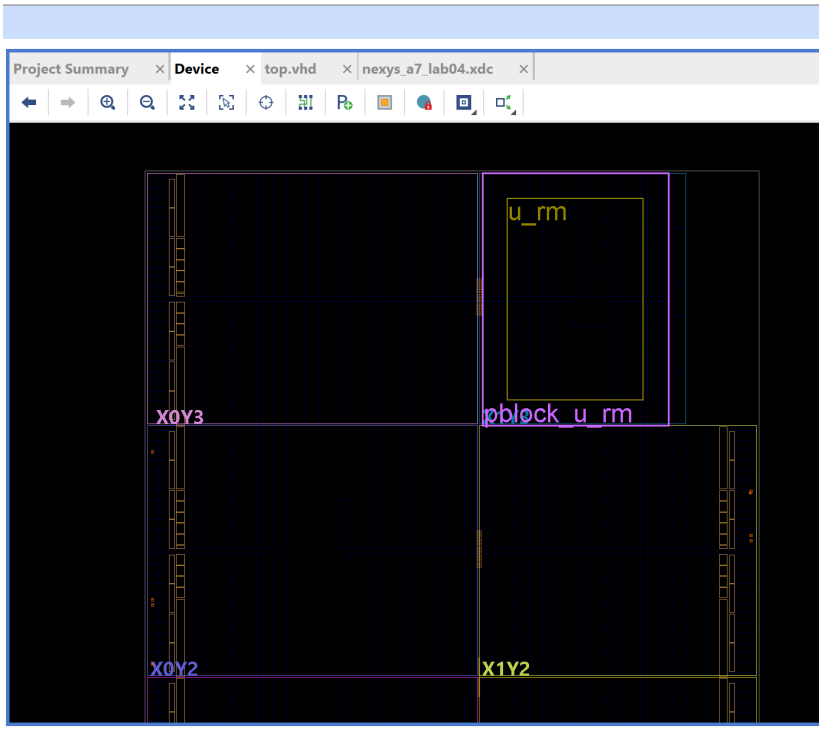


Figure 2 – Floorplan with pblock_u_rm in clock region X1Y3.

4. Implementation results

Run (configuration)	Module	LUTs	Registers	WNS	WHS	Power
impl_1 (config_1)	rm_a	1,634	1,764	1.861 ns	0.035 ns	0.232 W
child_0_impl_1 (config_2)	rm_b	1,650	1,764	1.861 ns	0.036 ns	0.232 W
child_1_impl_1 (config_3)	rm_c	1,634	1,764	1.861 ns	0.036 ns	0.232 W

- The static region is identical in all three runs: 1,634 LUTs and 1,748 registers are marked *fixed* in the child runs, and the setup slack is exactly 1.861 ns everywhere. Only the partition was placed and routed again.
- Each module adds 16 registers. rm_b also needs 16 LUTs for the inverters; rm_a and rm_c are only wiring, so they need no LUTs.
- All 3,119 nets fully routed, 0 routing errors. Remaining methodology warnings (TIMING-18 for the 19 slow board I/O, LUTAR-1 and TIMING-9 inside the reset/debug IP) are the same as in Lab 03.

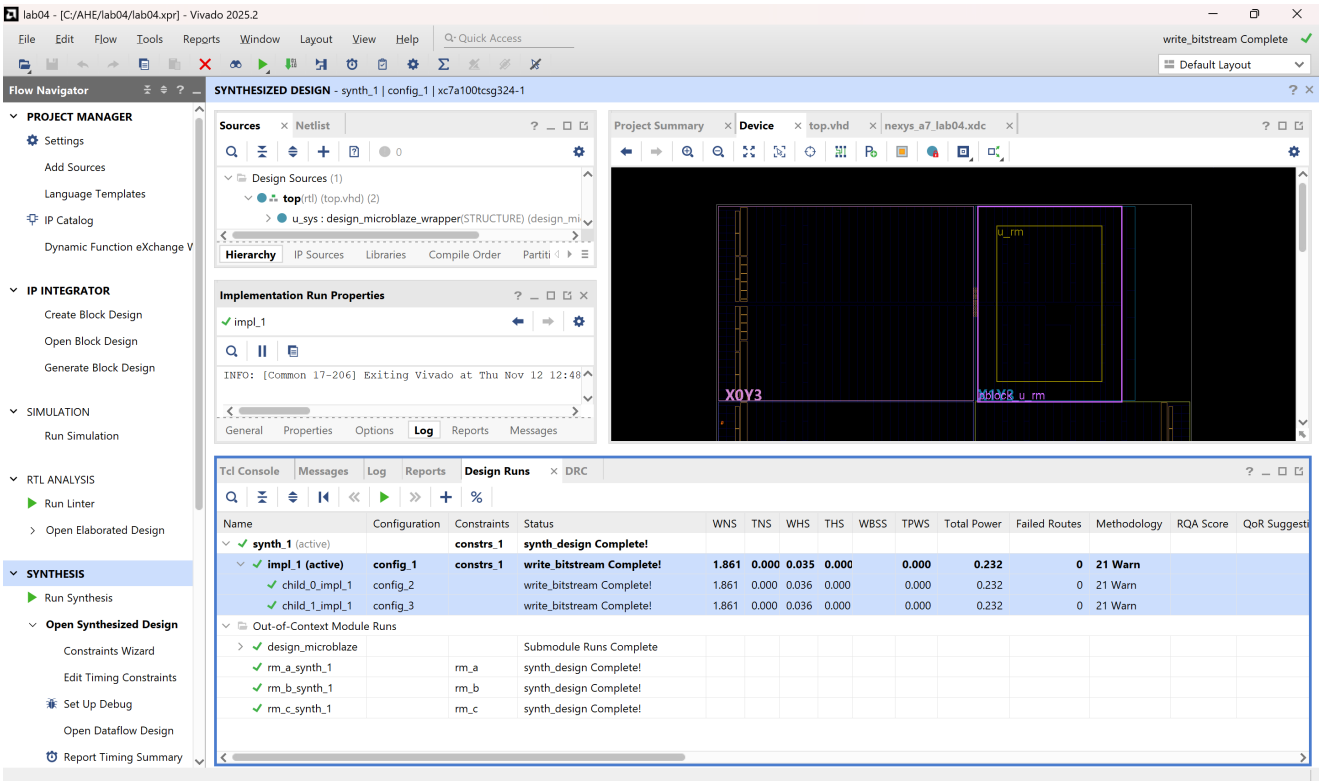


Figure 3 – Design Runs: parent impl_1 and the two child runs, all with write_bitstream complete.

5. Bitstreams

File	Generated by	Size (bytes)	Share	Configures
full_A.bit	impl_1 (top.bit)	3,825,904	100 %	Whole device: static region + rm_a
partial_A.bit	impl_1 (u_rm_rm_a_partial.bit)	799,693	20.9 %	Pblock only: rm_a
partial_B.bit	child_0_impl_1 (u_rm_rm_b_partial.bit)	799,693	20.9 %	Pblock only: rm_b
partial_C.bit	child_1_impl_1 (u_rm_rm_c_partial.bit)	799,693	20.9 %	Pblock only: rm_c

The partial files carry PARTIAL=TRUE in their header and have different checksums, so they contain different module logic in an identical set of frames.

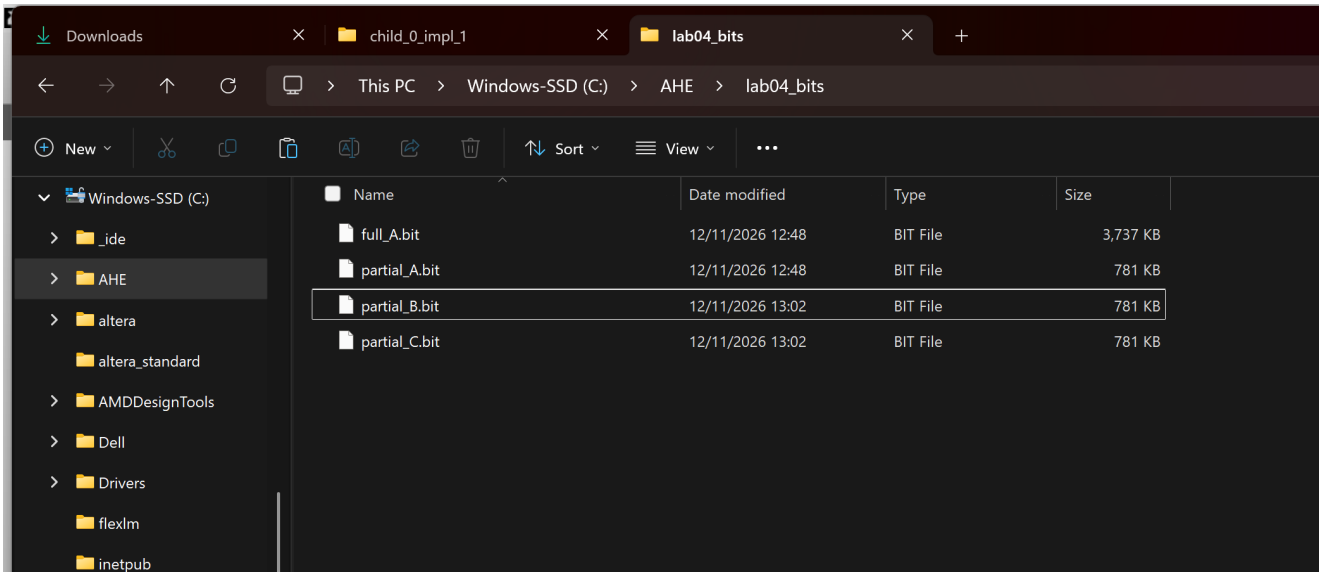


Figure 4 – Full and partial bitstreams with their sizes.

6. Software

The application **pr_demo** runs bare-metal on microblaze_0. It does not know which module is loaded; it finds out by sending the test value 0x0001 and comparing the answer. Then it sends a 16-bit counter and prints input and output four times per second. The counter is never reset, so a restart of the processor would be visible immediately in the log.

```
static const char *identify(void) {
    Xil_Out32(GPIO_BASE + DIN, 0x0001);              /* test value         */
    usleep(100);                                     /* module is registered */
    u32 r = Xil_In32(GPIO_BASE + DOUT) & 0xFFFF;
    if (r == 0x0001) return "RM A (pass-through)";
    if (r == 0xFFFE) return "RM B (invert)";
    if (r == 0x0100) return "RM C (swap bytes)";
    return "unknown";                                /* e.g. during the swap */
}
/* main loop: identify, write cnt to DIN, read DOUT, print, cnt++, usleep(250000) */
```

7. Demonstration on the Nexys A7

- Hardware Manager → Program Device with **full_A.bit**; pr_demo started from Vitis; serial monitor on COM6, 9600 baud.
- With the application running: Program Device with **partial_B.bit**, then **partial_C.bit**, then **partial_A.bit**.
- Every log was checked line by line with a script: the counter must increase by 1 per line, and each output must match the function of the module that is named in the same line.

```
loaded: RM A (pass-through)   in=004A out=004A
loaded: unknown   in=004B out=4CCE              <- partial_B.bit being loaded
loaded: unknown   in=004C out=FFCF
loaded: RM B (invert)   in=004D out=FFB2
   ...
loaded: RM B (invert)   in=00A8 out=FF57
loaded: unknown   in=00A9 out=FFF7              <- partial_C.bit being loaded
loaded: unknown   in=00AA out=D7FF
loaded: RM C (swap bytes)   in=00AB out=AB00
   ...
loaded: RM C (swap bytes)   in=00FD out=FD00
loaded: unknown   in=00FE out=FDF7              <- partial_A.bit being loaded
loaded: RM A (pass-through)   in=00FF out=00FF
```

Log	Lines	Counter range	Swaps	Wrong outputs
log_A_only.txt	337	continuous	–	0
log_A_to_B.txt	415	continuous	A → B	0
log_A_B_C.txt	1,034	0x04CD – 0x08D6	A → B → C	0
log_full_cycle_A_B_C_A.txt	336	0x0000 – 0x014F	A → B → C → A	0

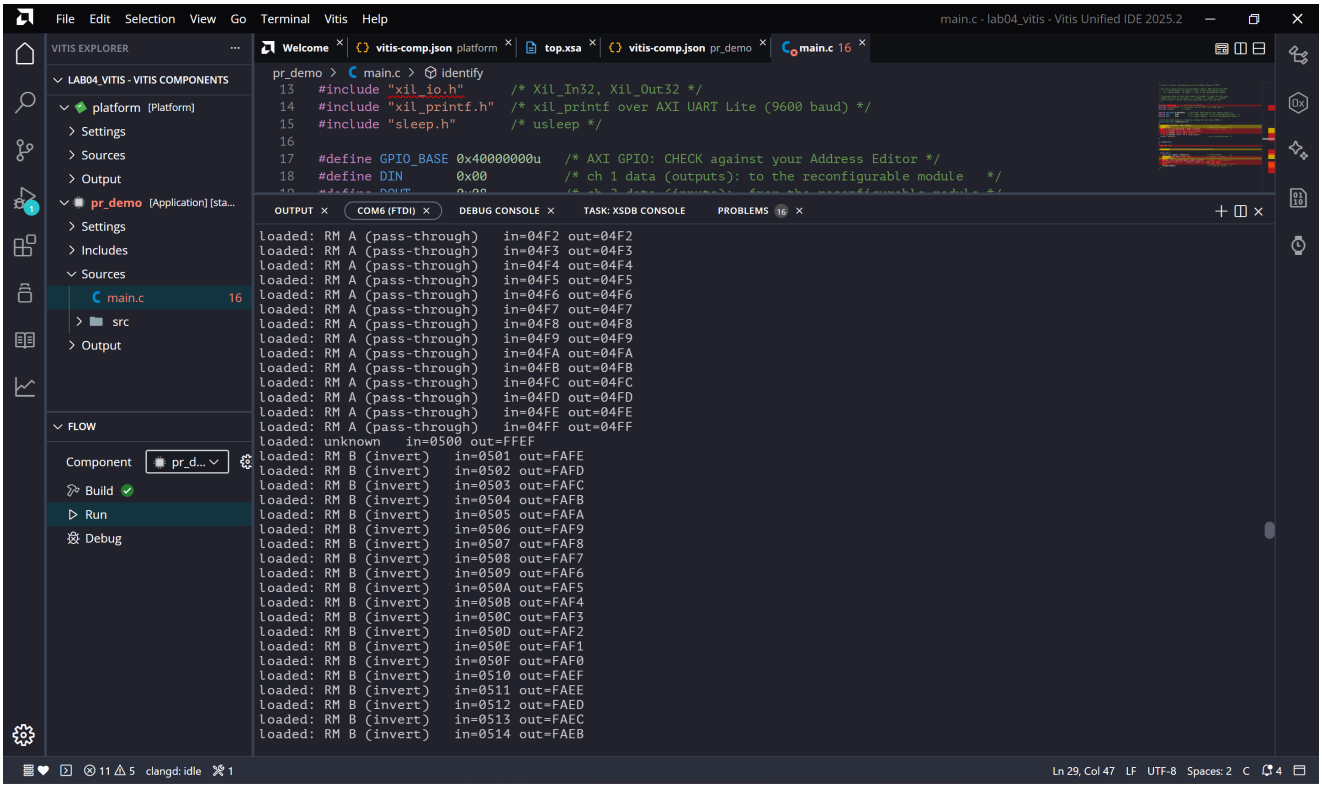
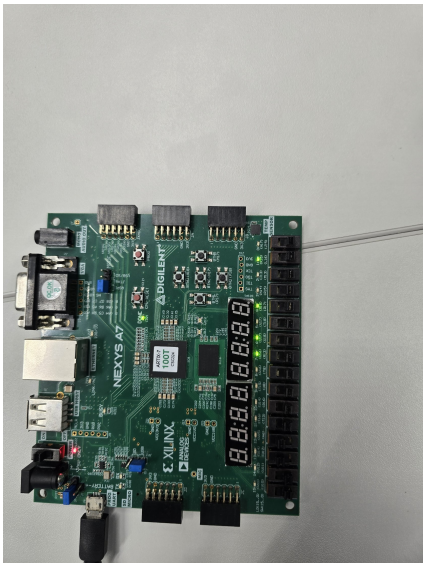
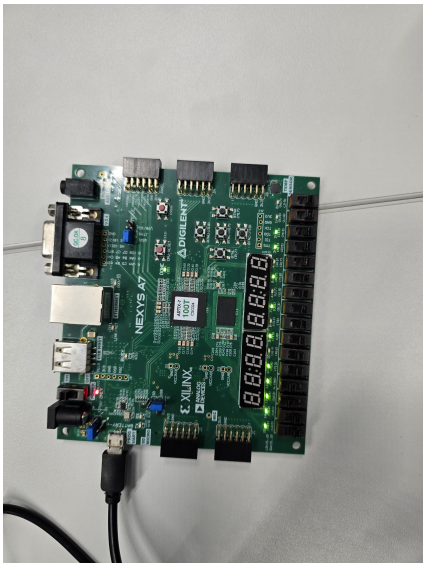


Figure 5 – Serial monitor in Vitis during the swap from module A to module B.



RM A – LEDs show the counter



RM B – inverted counter, upper LEDs on



RM C – counter moved to the upper byte

Figure 6 – LD0–LD15 show the output of the loaded module. After the full cycle the board showed LD5 + LD8 = 0x0120, matching the log line “RM A in=0120 out=0120”.

8. Answers to the questions of Task 4

How large is the full bitstream and how large are the partial ones?

The full bitstream is 3,825,904 bytes (about 3.65 MB). Each partial bitstream is 799,693 bytes (about 781 KB), 20.9 % of the full one. A partial bitstream only contains the configuration frames of the Pblock, not the whole device.

Why do the three partial bitstreams have the same size?

All three use the same Pblock, so they write the same set of configuration frames. Every frame inside the rectangle is written, also the frames a module does not use. The size depends only on the Pblock area, not on the logic: a pass-through, an inverter and a byte swap give exactly the same size. The contents (and checksums) differ.

What happened to the terminal output during the swap?

The program kept printing and the counter kept counting; no line was lost and nothing restarted. For 1–2 lines per swap the module was reported as *unknown*, with mixed values such as 0x4CCE or 0xFFCF: while the frames are being written, the outputs of the partition are undefined. With about 0.3 s per loop (250 ms delay plus the UART output), the swap over JTAG took roughly 0.3–0.6 s. The next line already showed the new module working correctly. A DFX Decoupler would hold the interface at a defined value during this time.

Why was the MicroBlaze not affected?

The MicroBlaze, its memory, the UART, the GPIO and the clocking all lie in the static region, outside the Pblock. A partial bitstream only writes frames inside the Pblock, so the configuration of the processor is never touched and its registers and memory keep their values. The identical static utilization and timing of all three runs show that the static region was locked and reused. (One processor halt seen during testing came from the Vitis debugger and the Hardware Manager sharing the JTAG cable, not from the reconfiguration.)

When would you not use reconfiguration?

When all functions fit into the device at the same time – here a multiplexer with a select register would be smaller and switch in one clock cycle. When switching must be fast: a swap takes about 0.3 s over JTAG and still milliseconds with ICAP. When the outputs must never be undefined, unless a decoupler is added. When the area overhead is too high: the Pblock reserves 1,500 slices plus BRAM, DSP and PCIE sites for a module that uses 16 flip-flops. And when the extra effort does not pay off: floorplanning, DRC fixes, one implementation run per configuration and testing every combination.

9. Problems encountered and solutions

Problem	Cause	Solution
Old wrapper checkpoint in Utility Sources	design_microblaze_wrapper.dcp left over from Lab 03	Checkpoint removed, wrapper regenerated
Wrapper showed rm_din_tri_i (input)	GPIO still had the Lab 03 direction	IP configuration: GPIO all outputs, GPIO2 all inputs
GPIO port names swapped	Ports renamed in the wrong order	Renamed rm_dout first, then rm_din
No Make External for rm_clk	Pin already connected to the clock net	Create Port (Ctrl+K) and connected it to the clock
Create Partition Definition missing	DFX not enabled in the project	Tools → Enable Dynamic Function eXchange
place_design error	Implementation started before the Pblock existed	Pblock drawn, implementation runs reset
Pblock too small / DRC HDPR-45	Pblock not full region height; PCIE site in the column excluded	Pblock extended to Y150–Y199; PCIE_X0Y0 added with resize_pblock
Terminal stopped after a later swap	Vitis debugger and Hardware Manager share the JTAG cable	CPU reset and restart; the full cycle then ran without a halt

Problem	Cause	Solution
False errors in the PROBLEMS tab	clangd indexer does not know the MicroBlaze GCC flags	Ignored – the mb-gcc build finished without errors

10. Summary

Static region. The part of the design that never changes: MicroBlaze, local memory, UART, GPIO, clock and reset. It is configured once with the full bitstream, implemented once in the parent run and locked; it keeps running during every swap.

Reconfigurable partition. A reserved area (`pblock_u_rm`, clock region X1Y3) with a fixed interface – `clk`, `din[15:0]`, `dout[15:0]`. It defines where a module may be placed and which frames a partial bitstream rewrites. Its size, not its contents, sets the partial bitstream size.

Reconfigurable modules. Different designs with identical ports that can occupy the partition: rm_a (pass-through), rm_b (invert) and rm_c (swap bytes). Each one is implemented in its own configuration against the same locked static region.

Partial bitstreams. Files that rewrite only the frames of the partition (799,693 B instead of 3,825,904 B). Loading one at run time changes the function of the hardware within a fraction of a second without stopping the processor – demonstrated with a complete A → B → C → A cycle and an unbroken counter.